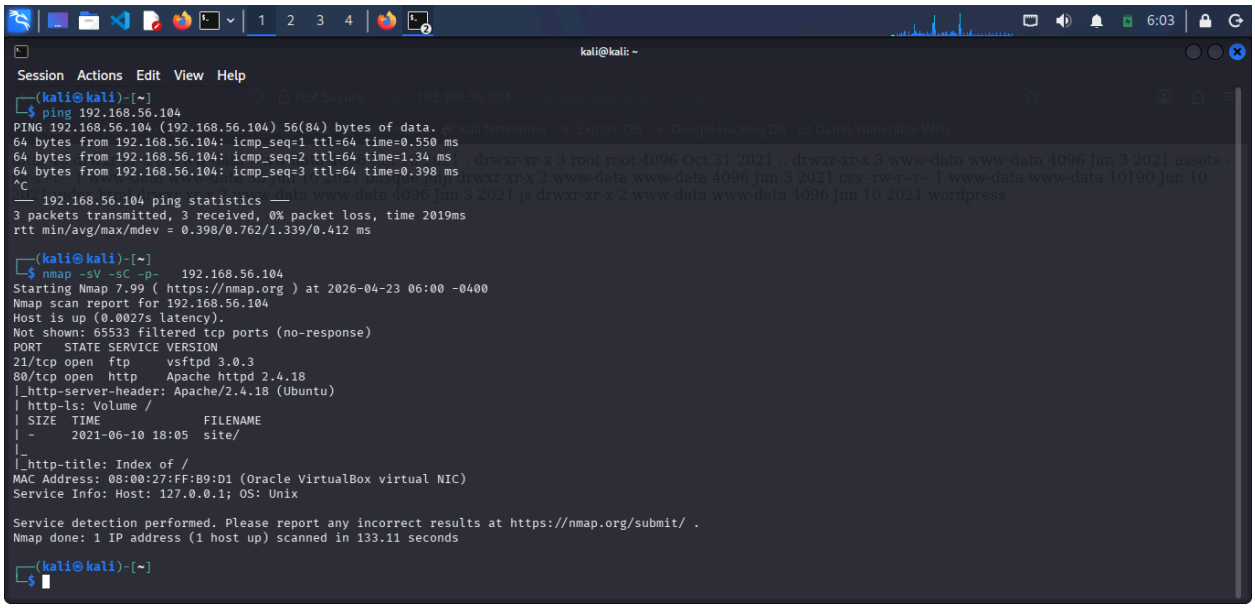
RELAZIONE TECNICA PENETRATION TEST: JANGOW-01

1. Fase di Information Gathering (Ricognizione)

Per prima cosa, ho dovuto capire cosa "offriva" il server bersaglio. Ho usato **nmap** per mappare i servizi aperti sull'IP **192.168.56.104**.

- **Comando: nmap -sV -sC -p- 192.168.56.104**
- **Cosa ho scoperto:** Il server ha la porta **21 (FTP)** e la porta **80 (HTTP)** aperte. Gira su Ubuntu e usa **Apache 2.4.18**. La presenza di FTP è un ottimo segno perché spesso èuna porta d'ingresso se si trovano le credenziali.
- 

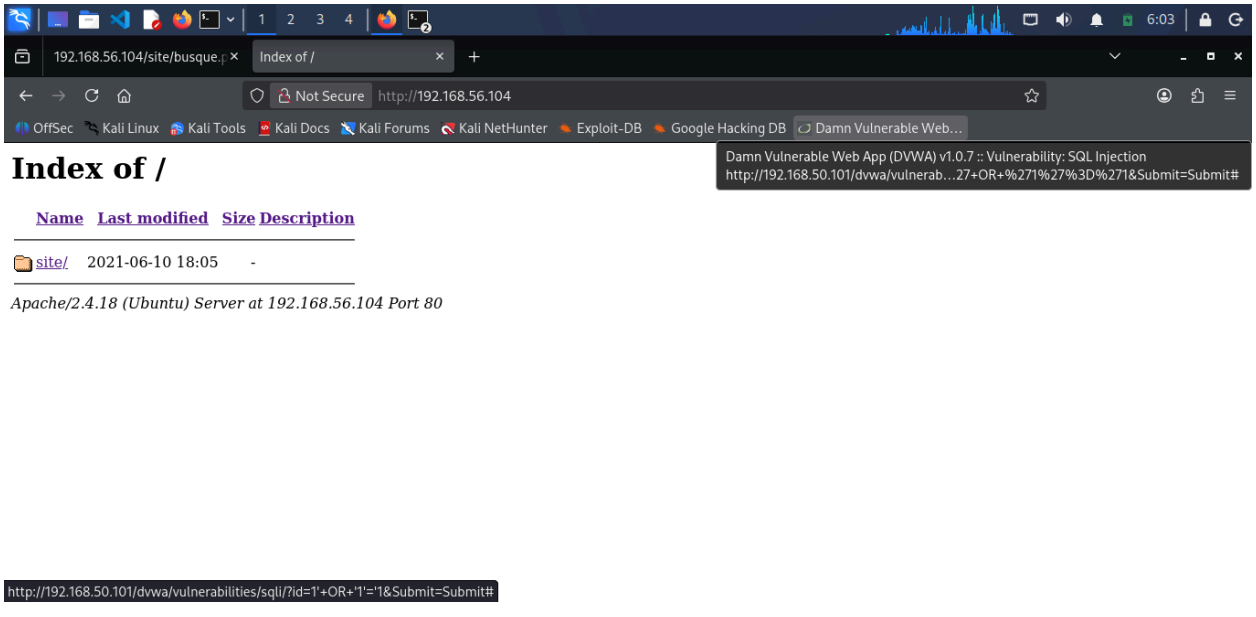
---

2. Analisi della Vulnerabilità: RCE via Parameter Injection

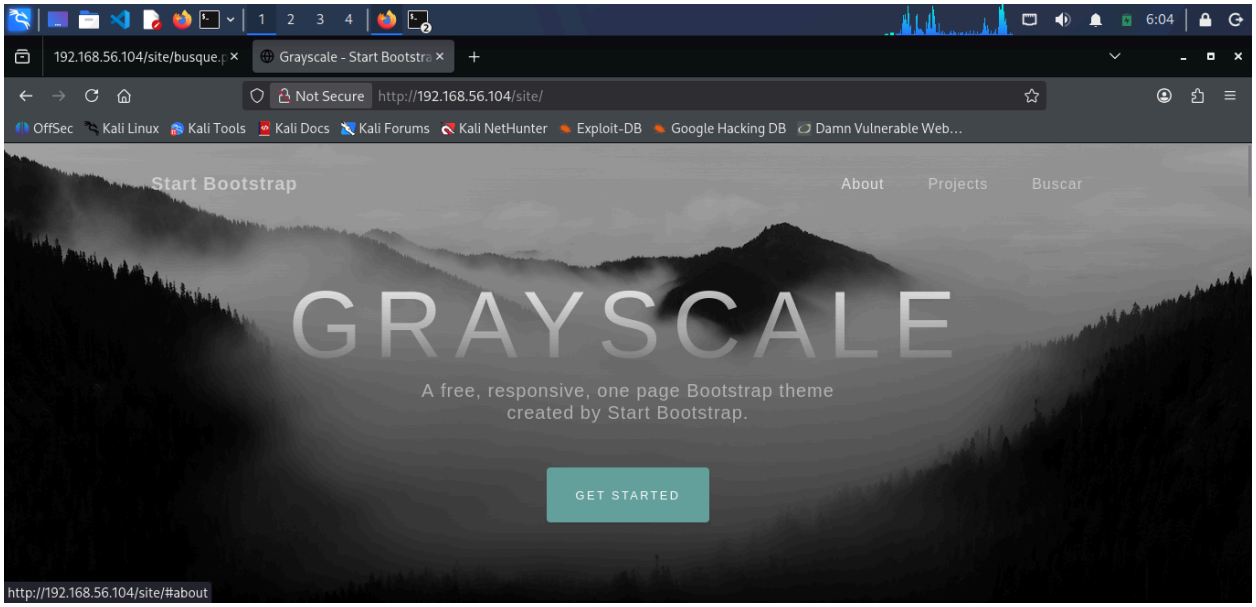
Navigando sul sito, ho notato che la pagina "Buscar" usa uno script PHP chiamato **busque.php.** La cosa fondamentale è il parametro **?buscar=** nell'URL.

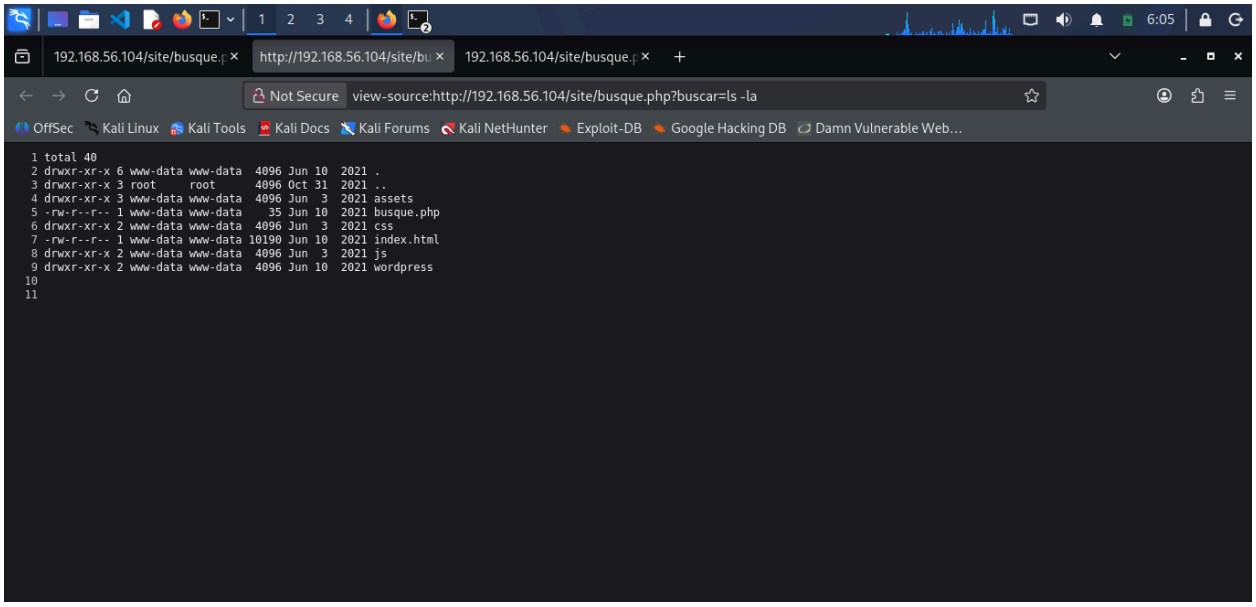
Ho capito subito che il programmatore ha fatto un errore gravissimo: tutto quello che scrivi dopo **?buscar=** viene passato direttamente alla funzione system() o exec() di PHP senza filtri. Questa si chiama **Remote Code Execution (RCE)**.

- **Test:** Ho scritto http://192.168.56.104/site/busque.php?buscar=ls -la
- **Risultato:** Il server mi ha risposto con l'elenco dei file. Le scritte evidenziate che vedi (tipo drwxr-xr-x) indicano i permessi dei file. Se posso fare ls, posso fare quasi tutto.



http://192.168.50.101/dvwa/vulnerabilities/sqli/?id=1'+OR+'1'='1&Submit=Submit#



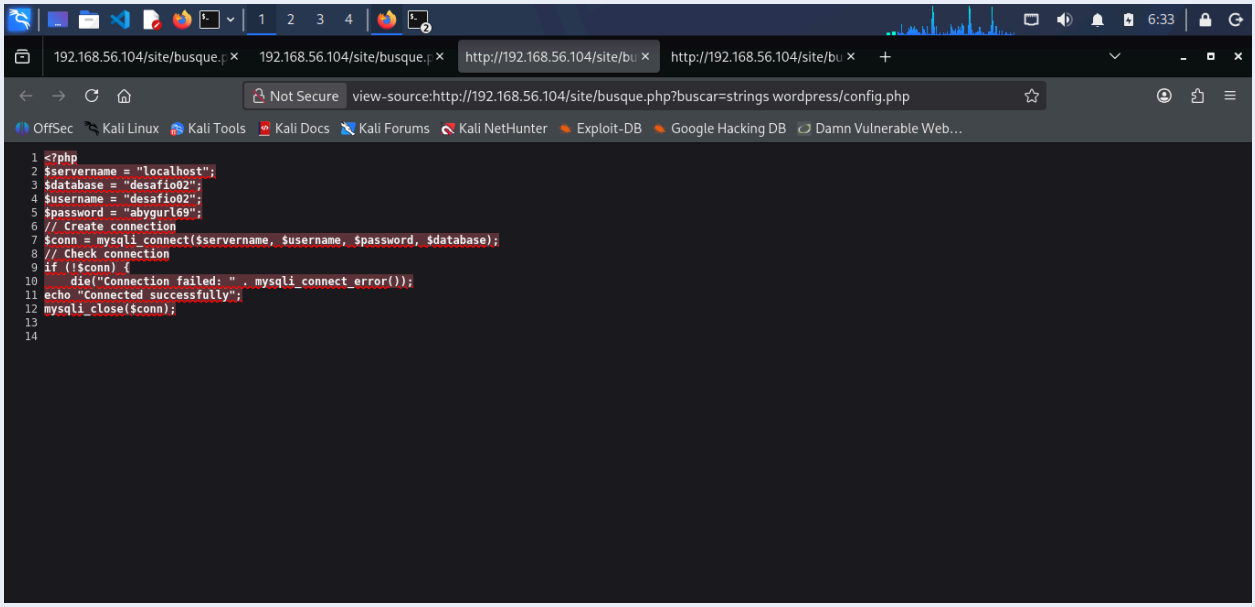


---

3. Esfiltrazione delle Credenziali

Visto che potevo eseguire comandi, ho cercato file sensibili. WordPress salva le password in chiaro in wp-config.php (o simili). Ho usato il comando strings per "pulire" il file e leggere solo il testo.

- **Comando iniettato: ?buscar=strings wordpress/config.php**
- **Cosa abbiamo trovato:** Come vedi nello screenshot, le scritte evidenziate in rosso sono le variabili di connessione.
  - **User:** desafio02
  - **Password:** abygurl69



Questa password è la nostra "chiave d'oro" perché spesso gli utenti usano la stessa password sia per il database che per il login di sistema.

4. Reverse Shell: Prendere il controllo del terminale

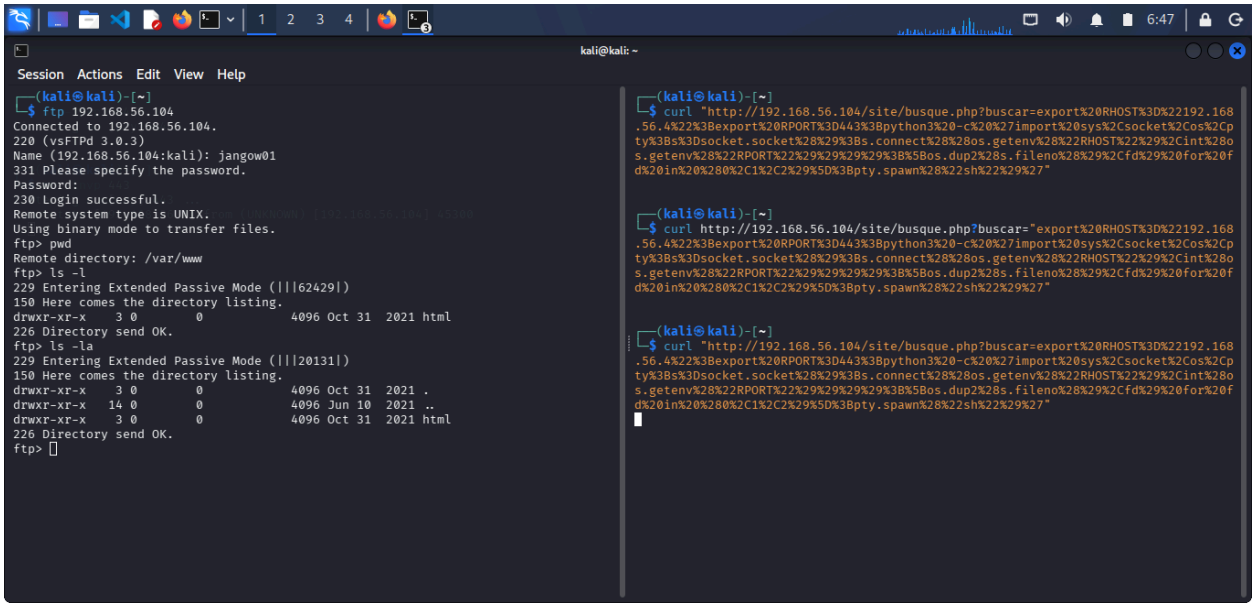
Lavorare dall'URL del browser è scomodo. Volevo una shell vera sulla mia Kali.

1. Sul mio terminale Kali ho lanciato **Netcat** in ascolto: **nc -lnvp 443.**
2. Sul browser, ho iniettato un comando **Python** nel parametro **?buscar=.**

Spiegazione tecnica dello script in rosso:

Lo script dice al server: *"Apri una connessione verso la Kali (192.168.56.4) sulla porta 443 e passami il controllo della tua shell /bin/sh"*.

Nello screenshot vedi a destra il comando **curl** che invia questa richiesta e a sinistra il mio terminale che riceve la connessione (**connect to [192.168.56.4] from...**). Ora sono "dentro".



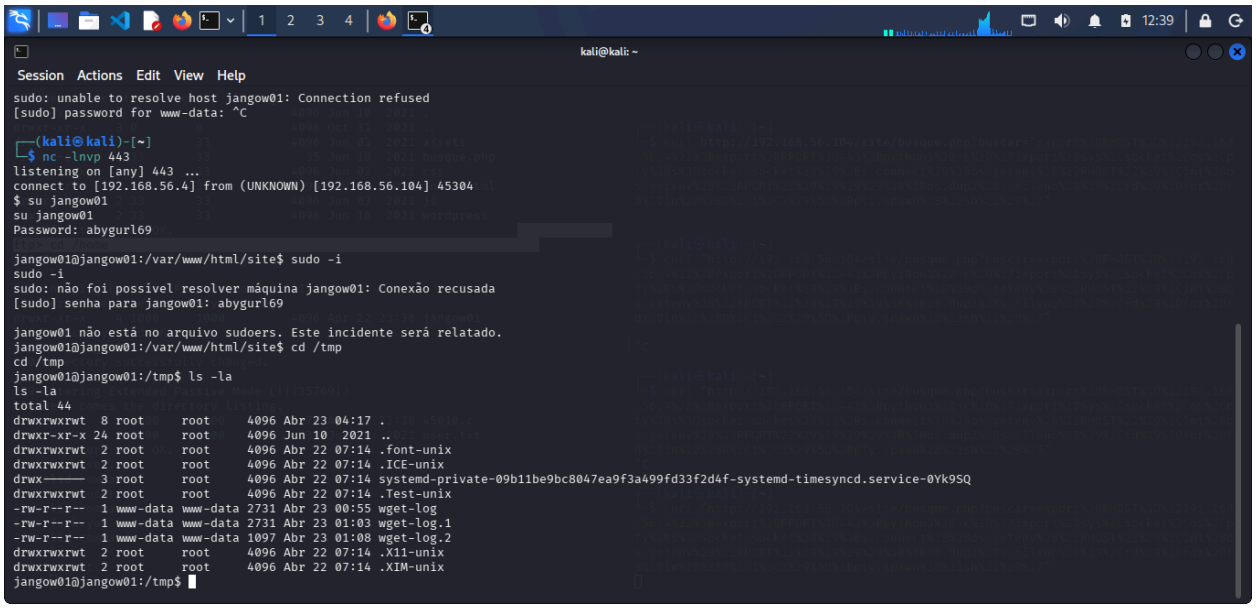
5. Escalation dei Privilegi (Verso Root)

Una volta dentro, ero l'utente **jangow01**. Ho provato a diventare Root con **sudo -i** usando la password **abygurl69**, ma come vedi nello screenshot, il sistema ha risposto che l'utente non è nel file sudoers.

Ho iniziato quindi l'analisi post-sfruttamento:

- Mi sono spostato in **/tmp,** perché è l'unica cartella dove qualsiasi utente può scrivere e caricare file.
- **Le scritte rosse nel terminale:** Indicano file con permessi speciali o log di download **(wget-log).**

- **L'obiettivo:** Sfruttare la versione vecchia del Kernel (Ubuntu 16.04) per caricare un exploit (come "OverlayFS") e ottenere i privilegi massimi di **ROOT**.



CONCLUSIONE

Il server è stato bucato perché il parametro **?buscar=** permetteva l'iniezione di comandi. Questo ci ha permesso di leggere la password abygurl69 e di scalare il sistema fino ad ottenere una shell remota. Per proteggere il server, il parametro buscar dovrebbe accettare solo lettere e numeri, bloccando simboli come ;, & o comandi di sistema.